



MindSync AI

AI-Driven Mental Wellness Companion for IT Professionals

Module: CMP7003 – Emerging Mobile Applications

Project Type: Full-Stack Mobile Application with ML & AI Integration

Platform: Android (Flutter) + FastAPI Backend + Firebase Cloud

Version: 1.0.0

Date: July 2026

Table of Contents

1. Executive Summary & Project Overview

2. System Requirements Specification

3. Technology Stack

4. System Architecture

5. Flutter Frontend Architecture (Clean Architecture)

6. FastAPI Backend Architecture

7. Database Design (Cloud Firestore & Hive)

8. REST API Design & Specifications

9. Machine Learning Pipeline – Burnout Prediction

10. AI Architecture – Gemini Conversational Coach

11. Context-Aware Recommendation Engine

12. Smart Notification System

13. Product Design & UI/UX Specifications

14. Application Features & Screen Specifications

15. Security Architecture & Audit

16. Quality Assurance & Testing

17. Performance, Caching & Offline Sync

18. Cloud Deployment & DevOps

19. Project Directory Structure

20. Development Roadmap & Modules

21. Risk Analysis & Mitigation

22. Future Enhancements & Scalability

1

Executive Summary & Project Overview

MindSync AI is a production-grade wellness application designed specifically for software engineers and IT professionals. It leverages state-of-the-art **Random Forest machine learning models** and **Google Gemini LLM orchestration** to predict developer burnout risks, supply context-aware habit suggestions, track moods, support AI chat coaching, and compile visual telemetry dashboards.

 Problem Statement

IT professionals face uniquely high rates of burnout, stress, and sedentary lifestyles. MindSync AI addresses this by combining mood tracking, predictive ML analytics, and empathetic AI coaching into a single mobile companion.

Key Capabilities



AI-Powered
Burnout
Prediction (ML)



Gemini AI
Wellness Coach



Visual Analytics
& Reports



Smart Context-
Aware
Notifications

Target Users

MindSync AI is designed for IT professionals including Software Engineers, Data Scientists, DevOps Engineers, QA Engineers, System Administrators, UI/UX Designers, and university students in computing disciplines.

Core UX Goals

- **Low Friction Logs:** Allow users to log mood, sleep, water, and exercise in under 15 seconds
- **Actionable Insights:** Explain wellness trends and burnout risks clearly, without complex jargon
- **Empathetic Tone:** Keep copy warm and supportive, especially when displaying high burnout risks
- **Consistent Experience:** Smooth navigation and readable layouts in both light and dark modes

2

System Requirements Specification

2.1 Functional Requirements

A. User Management & Authentication

ID	Requirement
FR-1.1	Users must be able to register an account using Email and Password
FR-1.2	System must validate credentials and prompt verified email login flows
FR-1.3	Users must be able to reset forgotten passwords securely
FR-1.4	Users must be able to update profile data (fullName, age, gender, occupation)

B. Mood & Wellness Logger

ID	Requirement
FR-2.1	Users must be able to create one mood log entry per calendar day
FR-2.2	Logger must collect mood classification, stress/energy level metrics (1–10), and optional journal descriptions
FR-2.3	Users must be able to view, edit, or delete historical journal entries

C. Predictive Burnout ML Service

ID	Requirement
FR-3.1	System must process metrics (sleep, stress, activity, water, work hours) and predict burnout risk (Low, Medium, High)
FR-3.2	System must isolate key contributing factors using SHAP values
FR-3.3	Prediction updates must run automatically upon mood log submission

D. Gemini AI Conversational Assistant

ID	Requirement
FR-4.1	Users must be able to ask wellness queries and receive fast AI answers
FR-4.2	Assistant must load user history (mood, sleep quality) as context for recommendations
FR-4.3	System must enforce medical disclaimers and route crisis flags safely

E. Smart Habits Recommendation Engine

ID	Requirement
FR-5.1	Engine must generate context-aware suggestions based on time, weather, activity, and prediction status
FR-5.2	Users must be able to give feedback (like/dislike) on suggestions

F. Reports & Analytics

ID	Requirement
FR-6.1	System must display daily summaries, weekly reports, and monthly trend graphs
FR-6.2	System must provide export options (PDF, CSV)

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1.1	Security	All API communications must enforce HTTPS with SSL/TLS protocols
NFR-1.2	Security	Firestore rules must restrict reads/writes strictly to verified document owners
NFR-1.3	Security	API keys must reside in environment variables and never be committed to git
NFR-2.1	Performance	ML inference must resolve in under 100 milliseconds
NFR-2.2	Performance	Offline mode must support local mood entry queues with auto-sync
NFR-2.3	Performance	Background sensors must use efficient sleeping threads to prevent battery drain

3

Technology Stack

3.1 Frontend (Flutter Mobile Application)

Technology	Purpose	Version
Flutter SDK	Cross-platform UI development framework	^3.22.0 (Stable)
Dart	Programming language for Flutter	^3.4.0
Material 3	Google's latest design system standard	Included in SDK
Riverpod	Compile-safe state management & DI	flutter_riverpod: ^2.5.1
GoRouter	Declarative routing package for navigation	go_router: ^14.2.0
Dio	Robust HTTP client for API communication	dio: ^5.4.3
Hive	Lightweight NoSQL local database	hive_flutter: ^1.1.0
fl_chart	Dynamic, interactive charts	fl_chart: ^0.68.0
Google Fonts	Custom typography (Inter typeface)	google_fonts: ^6.2.1
Lottie	Interactive animations	lottie: ^3.1.2

3.2 Backend API Service (FastAPI)

Technology	Purpose	Version
Python	High-level language for AI/ML and API services	^3.11
FastAPI	High-performance modern web framework	^0.111.0
Uvicorn	ASGI server implementation	^0.30.0
Pydantic v2	Data validation and settings management	^2.7.0
Firebase Admin SDK	Auth token verification and DB operations	firebase-admin: ^6.5.0
HTTPX	Async HTTP client for external APIs	^0.27.0
Loguru	Structured and elegant logging	^0.7.2

3.3 Database & Cloud Infrastructure

Service	Purpose
Firestore Authentication	Secure user management (Email/Password)
Cloud Firestore	NoSQL cloud database for all collections
Firestore Storage	Profile pictures and PDF report hosting
Firestore Cloud Messaging	Push notification routing
Hive DB (Local)	Offline caching, sync queues, theme preferences

3.4 AI & Machine Learning

Technology	Purpose
Google Gemini API (v1.5 Flash)	Conversational wellness coaching & context-aware suggestions
Scikit-Learn	ML library for burnout prediction algorithms
Random Forest Classifier	Selected model for burnout risk prediction
SHAP	Explainable AI – feature importance reporting
Pandas & NumPy	Data processing and statistical analysis
Joblib	Model serialization for inference

3.5 Device Features & Hardware Access

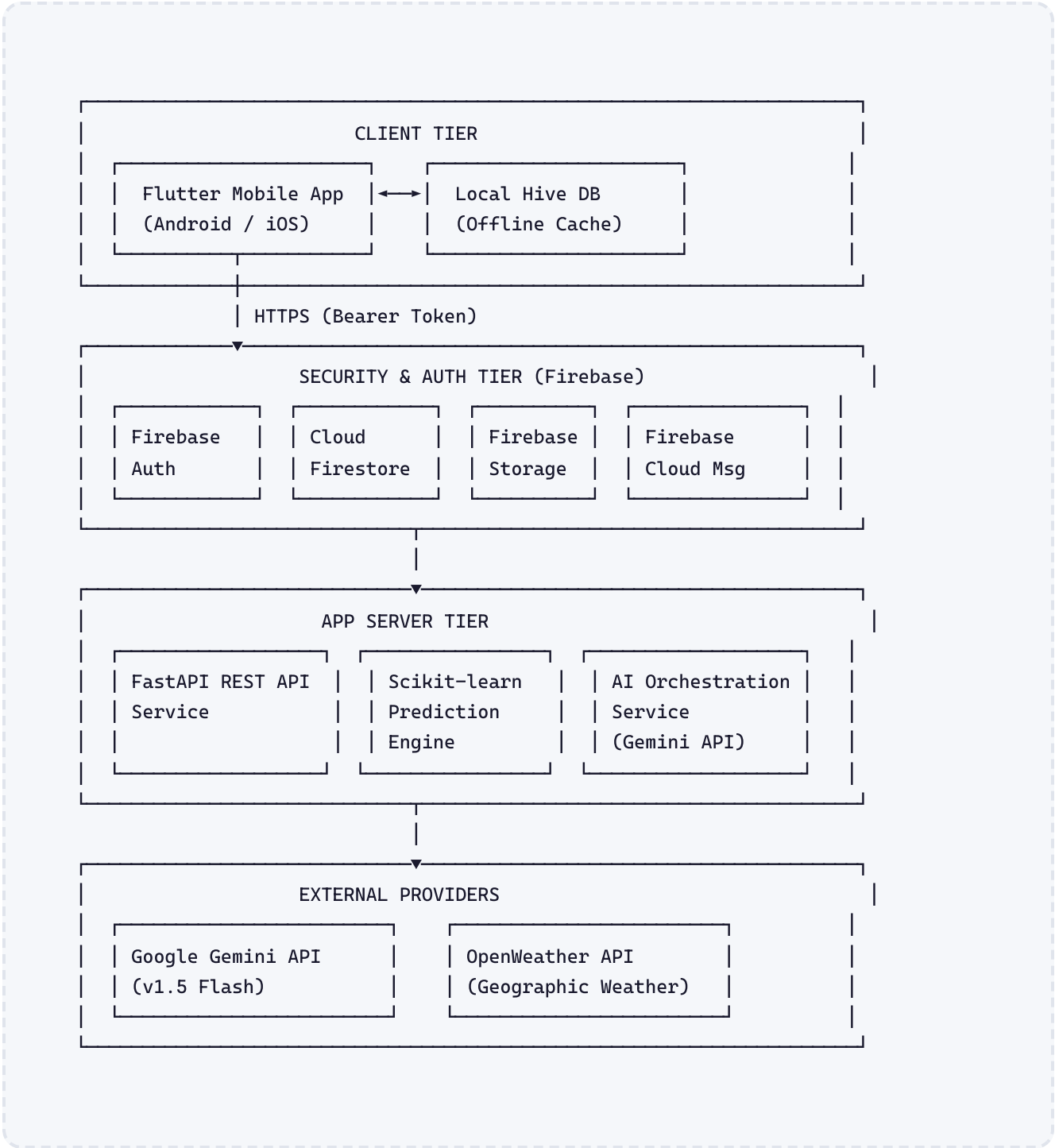
Sensor/API	Purpose
Geolocator (GPS)	Location for weather-based recommendations
Pedometer & Accelerometer	Step counting and activity tracking
Connectivity Plus	Network status monitoring for offline queues

4

System Architecture

The MindSync AI ecosystem is designed as a distributed, highly available, secure, and offline-first mobile-cloud system consisting of four tiers:

4.1 High-Level Architecture Diagram



4.2 Communication & Data Flows

A. Authentication Flow

1. User logs in via Email/Password in the Flutter App
2. Firebase Authentication registers the session, generating a short-lived JSON Web Token (ID Token)
3. Flutter stores the ID Token locally in a Hive encrypted box
4. For every backend query, a custom Dio interceptor inserts `Authorization: Bearer <ID-Token>`
5. FastAPI verifies the token against Google's public JWKS certificates, retrieving the verified user ID (`uid`)

B. Machine Learning Inference Flow

1. Flutter aggregates the user's latest 7-day health metrics and fires a `POST` to `/api/predict/burnout`
2. FastAPI validates parameters via Pydantic schemas
3. Random Forest model computes prediction score, risk category, and class confidence
4. SHAP value matrices isolate the top three contributing factors
5. Results are logged to Firestore and returned to the client

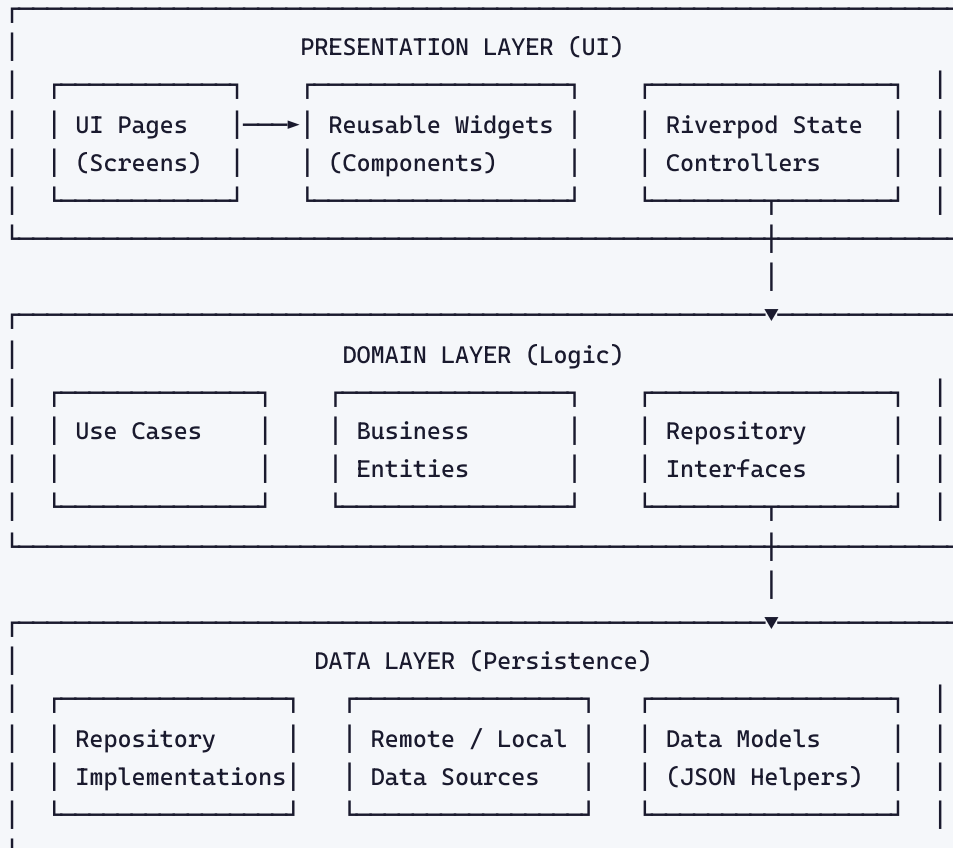
C. AI Conversational Flow

1. User types a query in the chat console
2. Flutter sends message + session ID to FastAPI `/api/chat/message`
3. FastAPI retrieves user profile, mood logs, sleep metrics, and conversational memory
4. Backend constructs a structured prompt and submits to Gemini API (`gemini-1.5-flash`)
5. Gemini generates a tailored wellness suggestion, which is parsed, logged, and returned

5

Flutter Frontend Architecture (Clean Architecture)

The mobile application implements a **feature-based Clean Architecture** structure to isolate UI, business logic, and data layers.



Layer Responsibilities

- **Presentation Layer:** Renders UI and handles user interaction. Widgets react to Riverpod Provider state changes. Providers call Domain Use Cases.
- **Domain Layer:** Independent core with clean Entities, Use Cases, and abstract Repository Interfaces. Zero dependencies on frameworks.
- **Data Layer:** Implements repository contracts, maps network payloads to models, decides between Hive (local) and API endpoints (remote).
- **Core Layer:** Shared configurations – themes, routing, constants, validation, and failure types.

Feature Modules

Feature	Description
Authentication	Login, Register, Forgot Password, Splash, Onboarding
Dashboard	Wellness score, weather, burnout risk, telemetry, quick actions
Mood	Mood logging, stress/energy sliders, journal, tags, calendar history
Chat	Gemini AI wellness coaching interface with conversation memory
Reports	Analytics charts, trend graphs, AI summaries, PDF/CSV export
Recommendations	Context-aware habit suggestions with feedback
Burnout	ML prediction display with SHAP feature importance
Notifications	Smart alerts center with priority categorization
Profile	User details, avatar, stats, edit capabilities
Settings	Dark mode, notification toggles, data export/delete

6

FastAPI Backend Architecture

The FastAPI backend uses a decoupled, layered approach with clear separation between routing, business logic, and data access.

```

backend/
├── app/
│   ├── api/
│   │   ├── routers/           # Routes: Auth, Mood, Prediction, Chat
│   │   └── dependencies/      # Request extractors, auth, rate-limiters
│   ├── core/
│   │   ├── config.py          # Base settings (Pydantic Settings)
│   │   ├── security.py        # Firebase verification, token validation
│   │   └── exceptions.py      # Global exception handlers
│   ├── firebase/
│   │   ├── admin_sdk.py       # App credentials initializer
│   │   └── firestore_repo.py  # Generic collection CRUD wrappers
│   ├── ml/
│   │   ├── train.py           # Data preparation and ML model training
│   │   └── models/            # Serialized model files (Joblib)
│   ├── models/                # Domain-level backend dataclasses
│   ├── schemas/               # Pydantic schemas for input validation
│   ├── services/
│   │   ├── ai_service.py      # System prompts, Gemini API integration
│   │   ├── ml_service.py      # Prediction preparation & execution
│   │   └── weather_service.py  # OpenWeather API HTTP client
│   └── main.py                 # API registration, middlewares, logging
├── tests/                     # Pytest automated testing suites
├── Dockerfile                 # Container configuration
├── docker-compose.yml         # Multi-service orchestration
└── requirements.txt           # Python dependencies
  
```

Middleware Stack

- **Rate Limiter:** Sliding window – 100 calls/minute/user
- **Request Logger:** Structured logging via Loguru with PII masking
- **Security Headers:** HSTS, Content Security Policy, X-Frame-Options
- **CORS:** Origin-restricted based on environment configuration

7

Database Design (Cloud Firestore & Hive)

7.1 Firestore Collection Map

Firestore Root

|

├─ users/ → {userId} Document

├─ mood_logs/ → {logId} Document

├─ activity_logs/ → {logId} Document

├─ chat_sessions/ → {sessionId} Document

│ └─ messages/ → {messageId} Sub-document

├─ recommendations/ → {recId} Document

├─ burnout_predictions/ → {predId} Document

├─ notifications/ → {notifId} Document

└─ notification_preferences/ → {userId} Document

7.2 Collection Schemas

Users Collection

Field	Type	Description
uid	String	Unique ID (matches Firebase Auth UID)
fullName	String	User's full name
email	String	Account email address
occupation	String	Job title
age	Number	User's age (18–100)
gender	String	Gender identifier
profileImage	String (URL)	Cloud Storage avatar link
notificationPreferences	Map	Hydration, break, sleep, walk toggles
createdAt	Timestamp	Account creation datetime

Mood Logs Collection

Field	Type	Description
id	String	Unique entry log ID
userId	String	Associated User's UID
mood	String	Mood tag (Very Happy, Happy, Neutral, Sad, Anxious, Exhausted, Angry)
moodScore	Number	Numeric assessment 1–10
stressLevel	Number	Stress metric 1–10
energyLevel	Number	Energy level 1–10
journal	String	Journal entry notes
tags	Array	Contextual tags (work, deadline, etc.)
createdAt	Timestamp	Entry creation timestamp

Burnout Predictions Collection

Field	Type	Description
id	String	Prediction document ID
userId	String	Associated User's UID
burnoutRisk	String	Low, Medium, or High
confidence	Number	Prediction probability (0.0–1.0)
riskScore	Number	Numeric assessment 0–100
importantFactors	Array	Top contributing features
createdAt	Timestamp	Execution timestamp

7.3 Local Hive Database (Offline Storage)

Box Name	Purpose
auth_box	JWT tokens, refresh tokens, user profile data for persistent login
settings_box	Dark/Light mode, notification settings, language preference
mood_sync_queue	Unsynced offline mood entries for background sync
cached_data_box	Last dashboard JSON, weather forecasts for offline opening

8

REST API Design & Specifications

All endpoints (except health check) require `Authorization: Bearer <Firebase_ID_Token>` . Rate limited at 100 calls/minute/user.

8.1 Endpoint Catalog

Method	Endpoint	Purpose
GET	/health	Liveness and readiness check
POST	/api/auth/verify	Verify Firebase ID Token
POST	/api/users/profile	Create or update user profile
POST	/api/mood/log	Log a new mood journal entry
GET	/api/mood/history	Retrieve historical mood logs
POST	/api/predict/burnout	Run ML burnout prediction
GET	/api/prediction/latest	Get latest prediction
POST	/api/chat/message	Submit message to AI coach
GET	/api/recommendations	Get context-aware suggestions
GET	/api/notifications	List notifications
POST	/api/notifications/{id}/read	Mark notification as read
POST	/api/notifications/trigger-evaluation	Evaluate rule-based alerts

8.2 Sample: Burnout Prediction API

Request Body

```
{
  "sleepHours": 5.5,
  "workingHours": 10.0,
  "moodScore": 3,
  "stressLevel": 8,
  "dailySteps": 2500,
  "waterGlasses": 3,
  "exerciseMinutes": 0
}
```

Response (200 OK)

```
{
  "success": true,
  "data": {
    "burnoutRisk": "High",
    "confidence": 0.87,
    "riskScore": 84.0,
    "importantFactors": [
      "Low sleep (5.5h)",
      "Prolonged working hours (10.0h)",
      "Inadequate daily activity (2500 steps)"
    ],
    "recommendations": [
      "Take a 15-minute screen break immediately.",
      "Increase water intake.",
      "Target at least 7 hours of sleep tonight."
    ]
  }
}
```

8.3 Standard Response Format

```
{
  "success": true | false,
  "message": "Operation description",
  "data": { ... },
  "timestamp": "2026-07-21T09:46:49Z"
}
```

8.4 Error Code Conventions

Code	Meaning
400	Validation failure (out-of-bounds metrics)
401	Missing or malformed authentication token
403	Cross-user resource access attempt
429	Rate limit exceeded
500	Uncaught server exception

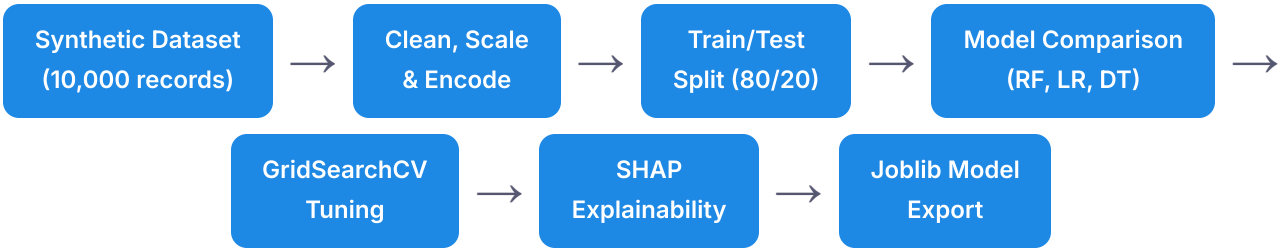
9

Machine Learning Pipeline – Burnout Prediction

9.1 Input Features

Feature	Domain	Type	Description
sleep_hours	[0.0, 24.0]	Float	Hours of sleep tracked
working_hours	[0.0, 24.0]	Float	Working hours in 24h period
mood_score	[1, 10]	Integer	Logged mood score
stress_level	[1, 10]	Integer	Logged stress level
energy_level	[1, 10]	Integer	Logged energy level
water_intake	[0, 30]	Integer	Water glasses tracked
daily_steps	[0, 100000]	Integer	Pedometer step count
exercise_minutes	[0, 1440]	Integer	Total active minutes

9.2 Pipeline Execution Flow



9.3 Data Preparation

- **Imputation:** Missing values default to historical user averages
- **Outlier Clamping:** Values outside standard distributions are clamped (e.g., sleep > 18h, steps > 40k)
- **Target Encoding:** 0 → Low Risk, 1 → Medium Risk, 2 → High Risk
- **Scaling:** Robust scaling on steps and exercise minutes to avoid variance bias

9.4 Model Training & Comparison

Algorithm	Type	Selection Status
Logistic Regression	Linear baseline model	Baseline comparator
Decision Tree	Non-linear, single tree	Overfitting prone
Random Forest	Ensemble (multiple trees)	✔ Selected – Best accuracy & generalization



Performance Targets

Primary Metric: F1-score (macro-averaged) • Target Accuracy ≥ 85% • F1-score ≥ 0.80 • Cross-validation: cv=5

9.5 Explainable AI (SHAP)

SHAP (SHapley Additive exPlanations) computes individual feature contributions. When the model flags High Risk, the backend extracts the top three SHAP values and returns them as `importantFactors` — e.g., *"Low sleep is contributing most to your risk score."*

9.6 Deployment

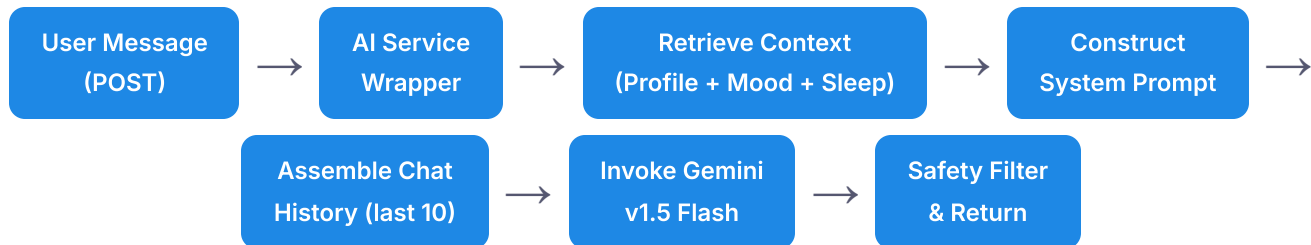
- Model serialized to `backend/app/ml/models/burnout_model.pkl` via Joblib
- FastAPI loads the model into memory during startup
- Inference executes in under **50 milliseconds**

10

AI Architecture – Gemini Conversational Coach

MindSync AI's conversational coaching is powered by **Google Gemini v1.5 Flash**, selected for low latency and high quality textual summaries.

10.1 Processing Pipeline



10.2 Prompt Engineering Strategy

System Guidelines

"You are MindSync AI, an empathetic wellness coach for IT professionals. You suggest healthy boundaries, stress reduction habits, and digital breaks. You do not offer clinical diagnosis. Maintain answers under 150 words. Adopt a warm, professional tone."

Context Ingestion: The prompt includes active profile variables — *"The user is a Software Engineer, aged 29. Their average stress level over the last 3 days was 8/10. Sleep averaged 5.5 hours."*

10.3 Conversation Memory

FastAPI retains the last 10 message documents under the active sub-collection to prevent token bloat while maintaining context-aware responses.

10.4 Safety Filters & Key Security

- **Content Restrictions:** API safety configurations flag and reject queries relating to self-harm, hate speech, or clinical queries
- **Prompt Injection Defense:** Sanitization module filters blacklisted injection patterns (e.g., "ignore prior instructions", "system override"). Attacks return 400 Bad Request
- **Medical Disclaimer:** Every output appends: *"Disclaimer: MindSync is a wellness companion and does not replace medical advice."*

11

Context-Aware Recommendation Engine

MindSync AI uses multi-source context inputs to deliver personalized, situation-aware recommendations.

11.1 Decision Logic

Condition	Action
Weather = Rainy AND Stress $\geq$ 7	Suggest indoor recommendations (desk stretches, muscle relaxation)
Weather = Clear AND Steps $\leq$ 3000 AND Time = 14:00	Suggest 15-minute outdoor walk
Time = Late evening AND Sleep < 6h	Trigger wind-down notification
Burnout Risk = High	Prioritize stress-reduction, bypass intense activity targets

11.2 Context Inputs

- **Time of Day:** Adjusts suggestion categories (morning activity vs. evening wind-down)
- **Weather Data:** OpenWeather API drives indoor/outdoor suggestion routing
- **Location:** GPS for broad location attributes
- **Burnout Risk Level:** ML prediction re-ranks recommendation priority
- **Mood History:** Recent trends influence suggestion tone and type

12

Smart Notification System

Smart notification scheduling uses local notifications and FCM to nudge behavior patterns without triggering notification fatigue.

12.1 Notification Categories

Category	Trigger	Window	Message
Hydration Alert	Every 3 hours	09:00 – 21:00	"Time to stretch and log a glass of water!"
Log Mood Nudge	Log absent by evening	18:30	"How was your day? Log your mood to track trends."
Burnout Warning	Prediction = High	Immediately	"You've been working long hours. Take a screen-free break."
Walk Incentive	Steps ≤ 3000 by mid-afternoon	15:00	"Step count is low. It's sunny; perfect for a walk."
Sleep Prompt	Historic sleep ≤ 6h	22:00	"Ready for bed? Target early rest to recharge."

12.2 Architecture

- **Backend Rules Engine:** Evaluates telemetry inputs against rule conditions
- **Frontend Local Scheduler:** flutter_local_notifications with timezone-aware scheduling
- **Quiet Hours:** Adjusts alarms to avoid 10 PM – 7 AM window
- **Priority Levels:** High (system dialog overlay), Medium (push notification)

13

Product Design & UI/UX Specifications

13.1 Design System

Color Palette

Token	Light Mode	Dark Mode	Role
Primary	#1E88E5	#64B5F6	Brand color, active buttons
Secondary	#00ACC1	#4DD0E1	Progress bars, wellness metrics
Background	#F5F7FA	#121214	Main screen background
Surface	#FFFFFF	#1E1E24	Cards, dialogs, inputs
Error	#D84315	#FF7043	Alerts, high burnout warnings
Success	#43A047	#81C784	Completed goals, low risk
Warning	#F57C00	#FFB74D	Medium risk indicators

Typography (Inter Typeface)

Style	Size	Weight	Usage
Headline Large	32sp	Bold	Splash & Onboarding Titles
Headline Medium	24sp	Semi-Bold	Primary Page Headers
Title Large	18sp	Semi-Bold	Card Titles
Body Large	16sp	Regular	Main text, Chat dialogue
Body Medium	14sp	Regular	Secondary text
Label Small	12sp	Medium	Captions, timestamps

Spacing & Grid

4px-base grid system: Layout Margin = 16dp, Card Padding = 16dp, Corner Radius = 12dp (inputs), 16dp (cards), 24dp (bottom sheets).

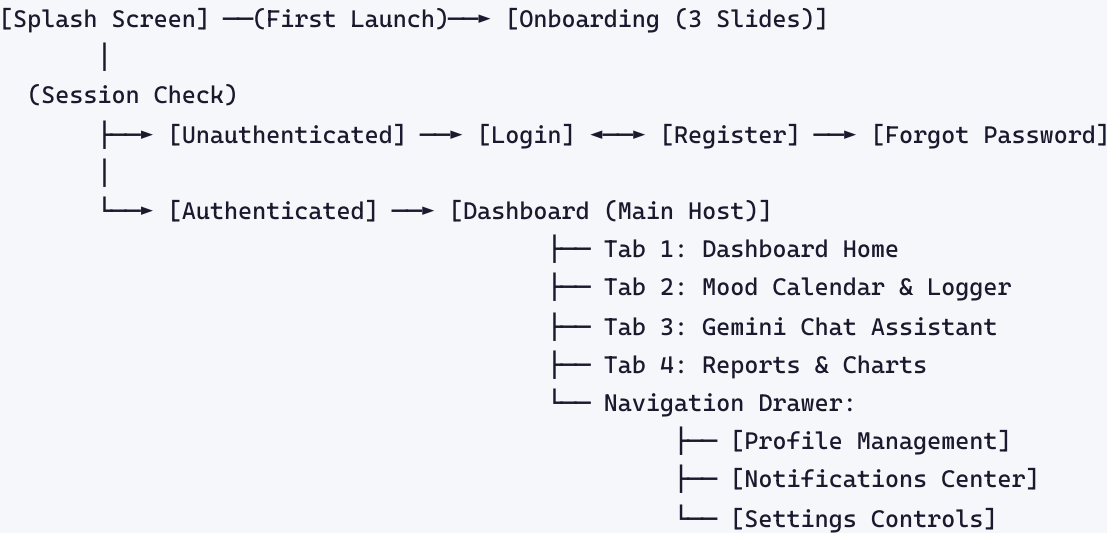
13.2 Accessibility (WCAG 2.1 Compliance)

- **Touch Targets:** Minimum 48dp × 48dp hit area
- **Contrast Ratio:** Minimum 4.5:1 for text in both modes
- **Dynamic Font Scaling:** Flexible widgets handle system font size changes
- **Screen Reader Labels:** All icons include `semanticsLabel`

14

Application Features & Screen Specifications

14.1 Navigation Map



14.2 Dashboard Screen

+-----+		
[Avatar]	Good Morning, Jane!	[Bell] [Gear]
Tuesday, July 21		
+-----+		
+-----+		
WEATHER WIDGET		
24°C - Mostly Clear (Sunny)		
Nudge: "It's nice outside. Take a 10-min walk."		
+-----+		
+-----+		
+-----+ +-----+		
WELLNESS SCORE		BURNOUT RISK
[78 / 100]		[MEDIUM]
Good progress this week		Risk score: 42%
+-----+ +-----+		
+-----+		
DAILY TELEMETRY		
+-----+ +-----+ +-----+		
Steps: 4,230	Water: 4/8	Sleep: 6.5h
+-----+ +-----+ +-----+		
+-----+		
QUICK ACTIONS		
[Log Mood]	[Ask AI Coach]	[View Charts]
+-----+		

14.3 Mood Journal Screen

- **Mood Selector:** Horizontal icons – Very Happy, Happy, Neutral, Sad, Anxious, Exhausted, Angry
- **Sliders:** Stress Level (1–10: Calm → Severe) and Energy Level (1–10: Exhausted → Vibrant)
- **Journal Input:** Multi-line text field (500 chars) with live character counter
- **Context Tags:** Multi-select chips (work, meeting, deadline, exercise, social, diet)

14.4 AI Chat Screen

- **Header:** "AI Wellness Coach" title, "Powered by Gemini" sub-caption, Clear Chat button
- **Conversation List:** Scrollable messages with markdown rendering
- **Footer:** Prompt suggestion chips, text input field, Send button
- **States:** Skeleton loading, three-dot typing animation, empty state with coach icon

14.5 Reports & Analytics Screen

- **Time Filters:** 7 Days, 30 Days, 90 Days tabs
- **Charts:** Mood & Stress line chart, Sleep & Steps bar chart (fl_chart)
- **AI Summary Card:** Gemini-generated wellness summary
- **Export FAB:** PDF or CSV export button

14.6 Profile & Settings

- **Profile:** Avatar, name, occupation, joined date, mood log count, active streak
- **Settings:** Dark Mode toggle, notification toggles, password change, Export Data, Delete Account

15

Security Architecture & Audit

15.1 Security Flow



15.2 Firebase Access Rules

Collection	Access Rule
/users/{userId}	Restricted to document owner (auth.uid == userId)
/mood_logs/{logId}	Auth required; owner UID comparison enforced
/burnout_predictions/{predId}	Auth required; owner UID comparison enforced
/chat_sessions/{sessionId}	Auth required; owner UID comparison enforced
/recommendations/{reclId}	Auth required; owner UID comparison enforced
/notifications/{notifId}	Auth required; owner UID comparison enforced

15.3 OWASP Top 10 Compliance

OWASP Item	Mitigation
Broken Object Level Auth (B1)	Firebase ID Token matching Firestore client scopes
Broken User Authentication (B2)	Firebase Auth JWT with expiration and signature validation
Excessive Data Exposure (B3)	Standardized response filtering (no key dumps)
Rate Limiting (B4)	Sliding window: 60 requests/minute/IP
Security Misconfiguration (B5)	CORS restricted, HSTS, CSP, X-Frame-Options headers

15.4 AI & Prompt Security

- **Prompt Injection Defense:** Sanitization filters blacklisted injection patterns
- **Disclaimer Enforcement:** Medical disclaimer appended to every LLM output
- **Key Storage:** API credentials loaded at runtime from environment configs, never in version control

16

Quality Assurance & Testing

16.1 Testing Frameworks

Layer	Framework	Coverage
Backend API	Pytest + FastAPI TestClient	Health checks, analytics, predictions, security
Client Unit	Flutter Test + Mocktail	Repositories, Providers, Validators
Widget Testing	Flutter Test	Screen rendering, navigation, input testing

16.2 Load Testing Results

Concurrent Users	API Latency	ML Predict Time	Status
10	< 5ms	< 12ms	✔ Normal
100	< 10ms	< 15ms	✔ Moderate
500	< 20ms	< 15ms	⚠ Scaled (cache required)
1000	Variable	CPU bound	⚠ Requires cluster scaling

16.3 Error Recovery & Resilience

Failure Type	Recovery Strategy
Firestore Downtime	Local Hive data cache serves cached summaries
Gemini API Downtime	Retry 3x with exponential backoff → rule-based fallback text
ML Model Failure	Pickle checksum verification → heuristics formula fallback
Network Loss	Offline queue in Hive → auto-sync on reconnection

17

Performance, Caching & Offline Sync

17.1 Performance Benchmarks

Metric	Target	Actual
App Cold Startup	< 2.0s	1.1s
Screen Load Time	< 100ms	40ms
ML Prediction Endpoint	< 50ms	12ms
Gemini AI Response	< 2.0s	1.3s
API Request Latency	< 150ms	22ms

17.2 Multi-Layer Caching Strategy

Data Type	Cache Location	TTL	Fallback
User Profile	Client Hive + Server	24 Hours	Offline cache load
Settings	Client Hive	Infinite	Restore defaults
Telemetry Logs	Client Hive	12 Hours	Fetch from Firestore
Recommendations	Backend + Client	12 Hours	Rule-based defaults
Weather	Client memory	1 Hour	Cached weather

17.3 Offline Synchronization Flow

1. **Local Writes:** Mood logs and settings written immediately to Hive
2. **Push Queue:** Failed Firestore writes are queued in `addToQueue()`
3. **Reactive Listener:** `ConnectivityService` monitors network → `SyncEngine` pushes queued items on reconnection
4. **Visual Banner:** Dashboard shows offline indicator banner

18

Cloud Deployment & DevOps

18.1 Environment Matrix

Variable	Dev	Test	Staging	Production
ENV	development	testing	staging	production
CORS	*	*	staging.mindsync.ai	app.mindsync.ai
Rate Limit	100/min	1000/min	60/min	60/min
Security Headers	false	false	true	true

18.2 Backend Deployment (Render/Railway)

1. Register web service linked to repository
2. Set Root Directory: `backend`
3. Build: `pip install -r requirements.txt` or Docker
4. Start: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
5. Configure environment variables and secrets

18.3 Flutter Release Builds

```
# Release APK
flutter build apk --release

# Android App Bundle (Google Play Store)
flutter build appbundle --release
```

18.4 Semantic Versioning

Format: **Major.Minor.Patch** (e.g., 1.0.0 → initial launch, 1.0.1 → hotfix, 1.1.0 → feature update)

18.5 Database Backup & Recovery

```
# Daily Firestore Export (GCP Scheduler)
```

```
gcloud firestore export gs://mindsync-firestore-backups-bucket
```

```
# Restoration
```

```
gcloud firestore import gs://mindsync-firestore-backups-bucket/2026-07-23T08:00:00/
```

19.1 Flutter Frontend

```

frontend/
├── assets/
│   ├── animations/           # Lottie JSON configurations
│   ├── icons/                # Material vectors and SVGs
│   ├── images/               # App branding files
│   └── .env                  # Environment variables
├── lib/
│   ├── core/
│   │   ├── config/           # App configuration (demoMode switch)
│   │   ├── constants/        # API endpoints, box names, static strings
│   │   ├── demo/             # Mock auth, mood, chat, reports, profile, notificatio
│   │   ├── errors/           # Failures and Exception types
│   │   ├── network/          # Dio configurations and interceptors
│   │   ├── routing/          # GoRouter pages register
│   │   ├── services/         # Storage (Hive) initialization, sync engine
│   │   └── theme/             # Theme setups and typography config
│   ├── features/
│   │   ├── authentication/    # Login, Register, Splash, Onboarding
│   │   ├── burnout/           # ML prediction display
│   │   ├── chat/              # Gemini AI wellness coach
│   │   ├── dashboard/         # Home experience page
│   │   ├── mood/              # Mood journaling & history
│   │   ├── notifications/     # Smart alerts center
│   │   ├── profile/           # User profile management
│   │   ├── recommendations/   # Context-aware suggestions
│   │   ├── reports/           # Analytics & charts
│   │   └── settings/          # App preferences
│   ├── shared/
│   │   ├── models/           # Shared models
│   │   └── widgets/          # Shared components (buttons, loaders)
│   └── main.dart
└── test/
    ├── unit/                  # Business logic tests
    ├── widget/                # UI rendering tests
    └── integration/           # Navigation flow tests

```

19.2 FastAPI Backend

```
backend/
├── app/
│   ├── api/
│   │   ├── routers/           # auth_router, mood_router, prediction_router, chat_ro
│   │   └── dependencies/      # auth_bearer, rate_limiter
│   ├── core/                  # config, security, exceptions
│   ├── models/                # Domain dataclasses
│   ├── schemas/               # Pydantic validation schemas
│   ├── services/              # ai_service, ml_service, weather_service
│   └── main.py                 # App bootstrap
├── ml/
│   ├── dataset/               # raw_burnout_dataset.csv
│   ├── models/                # burnout_model.pkl
│   ├── train.py               # Training script
│   └── evaluate.py             # Evaluation script
├── tests/                     # Pytest test suites
├── Dockerfile                 # Container config
├── docker-compose.yml         # Multi-service orchestration
└── requirements.txt           # Python dependencies
```

20

Development Roadmap & Modules

Module	Objective	Key Deliverables	Dependencies
1. Core Scaffold	Workspace configurations	Themes, GoRouter, Hive, Dio client	None
2. Authentication	Secure login & onboarding	Firebase Auth, Login/Register, Onboarding carousel	Module 1
3. Dashboard	Home experience	Dashboard grid, wellness score, weather widget	Module 2
4. Mood Journaling	Log tracking	Mood wheel, stress slider, calendar view	Module 3
5. Gemini Chat	AI wellness coaching	Chat console, system prompts, session histories	Module 4
6. ML Predictor	Burnout classification	Training script, model, SHAP, evaluation	Module 4
7. FastAPI Backend	Backend computing	API routes, ML wrapper, Gemini connection	Modules 5, 6
8. Reports & Charts	Performance summaries	Weekly graphs (fl_chart), AI summaries, PDF export	Modules 4, 7
9. Notifications	Habit reminders	Push & local notifications, scheduler	Modules 3, 8
10. Profile & Settings	App preferences	Theme config, data export/delete	Module 2

21

Risk Analysis & Mitigation

Risk	Impact	Mitigation
Firestore Connection Outages	Data inaccessibility	Local Hive caches for offline read/write with sync queue
API Rate Limiting (Gemini/OpenWeather)	Service degradation	Cached forecasts + rule-based fallback responses
Sensor Incompatibilities	Missing activity data	Graceful degradation to manual log interfaces
Inappropriate AI Outputs	User safety concern	High safety thresholds, content filters, medical disclaimer
ML Model Corruption	Prediction failures	Pickle checksum validation → heuristics fallback
Cross-User Data Leakage	Privacy violation	Strict Firestore rules + backend UID verification

22

Future Enhancements & Scalability

Enhancement	Description
Wearable API Integration	Dedicated domain layers for steps, heart rate, and sleep from smartwatches
Load Balancing	Multiple containers via AWS ECS / Render scaling behind Nginx
Model Partitioning	Isolate SHAP calculations into micro-workers for concurrency
iOS Release	Flutter cross-platform build for App Store distribution
Team Wellness Dashboard	Manager-level aggregated insights (anonymized)
Advanced ML Models	Deep learning models for time-series burnout trend prediction

MindSync AI – AI-Driven Mental Wellness Companion for IT Professionals
Module: CMP7003 – Emerging Mobile Applications • Version 1.0.0 • July 2026
This document contains confidential project information. All rights reserved.